

Python Notes Based on Teacher GitHub Repository (707)

1. Introduction to Python

What is Programming?

Programming means giving instructions to a computer to perform specific tasks. These instructions are written using programming languages.

Examples of tasks done using programming:

- Calculator
 - Banking software
 - Websites
 - Mobile applications
 - Games
 - Attendance systems
-

What is Python?

Python is a high-level, interpreted, and easy-to-learn programming language created by Guido van Rossum.

Python is one of the most popular programming languages because:

- Syntax is simple and readable
 - Easy for beginners
 - Large community support
 - Used in multiple fields
 - Requires less code compared to other languages
-

Features of Python

1. Easy Syntax

Python code looks simple and close to English language.

Example:

```
print("Hello")
```

2. Interpreted Language

Python executes code line by line using an interpreter.

Advantage:

- Easier debugging
 - Fast testing
-

3. Platform Independent

Python programs can run on:

- Windows
- Linux
- MacOS

without changing the code.

4. Open Source

Python is free to use.

5. Large Library Support

Python provides many built-in libraries for:

- Mathematics
 - Data Science
 - AI
 - Machine Learning
 - Web Development
-

Applications of Python

Python is used in:

Field	Example
Web Development	Websites
Data Science	Data Analysis
AI/ML	Chatbots
Automation	Automatic tasks
Game Development	Games
Cyber Security	Security tools

First Python Program

What is Python?

Python is a high-level, easy-to-learn programming language used for:

- Web Development
- Data Science
- Automation
- AI/ML
- Software Development

First Python Program

```
print("Hello World")
```

2. Data Types

What is a Data Type?

Data type defines the type of value stored in a variable.

Different types of data require different storage and operations.

Example:

- Integer stores whole numbers
- Float stores decimal numbers
- String stores text

- Boolean stores True/False values
-

Why Data Types are Important?

Data types help Python understand:

- What type of value is stored
- What operations can be performed
- How memory should be allocated

Example:

```
5 + 5
```

Python performs mathematical addition.

But:

```
"5" + "5"
```

Python joins the strings.

(From: 1_data_types.ipynb)

Integer (int)

Stores whole numbers.

Examples:

```
5  
10  
-20
```

Check datatype:

```
type(5)
```

Output:

```
<class 'int'>
```

Float

Stores decimal values.

Examples:

```
5.2  
10.5  
56.300
```

Check datatype:

```
type(5.2)
```

Output:

```
<class 'float'>
```

String

Stores text values.

Examples:

```
"Python"  
'Hello'
```

Check datatype:

```
type("Python")
```

Boolean

Stores True or False values.

Examples:

```
True
False
```

Important Difference

Data Type	Example
int	5
float	5.5
string	"Hello"
bool	True

3. Operators

What is an Operator?

Operators are symbols used to perform operations on values and variables.

Example:

```
5 + 2
```

Here:

- 5 and 2 are operands
- `+` is an operator

Types of Operators in Python

1. Arithmetic Operators

- 2. Comparison Operators
- 3. Logical Operators
- 4. Assignment Operators

Arithmetic Operators

(From: 2_operators_new_batch.ipynb and 3_operators.ipynb)

Arithmetic Operators

Operator	Meaning	Example
+	Addition	5 + 2
-	Subtraction	5 - 2
*	Multiplication	5 * 2
/	Division	10 / 2
//	Floor Division	19 // 2
%	Modulus	19 % 2
**	Power	5 ** 2

Examples

```
5 + 9
90 - 150
10 * 2
5 ** 2
10 / 2
19 // 2
```

Important Concepts

Division

```
19 / 2
```

Output:

```
9.5
```

Floor Division

```
19 // 2
```

Output:

```
9
```

String Concatenation

Joining strings together.

Example:

```
"Hello" + "Python"
```

Output:

```
HelloPython
```

Wrong Example:

```
"Hello" + 7
```

This gives an error because string and integer cannot be added directly.

4. Variables and Printing

(From: 3_operations_new.ipynb)

Variable

A variable stores data.

Example:

```
name = "Om"  
age = 20
```

Printing Variables

```
print(name)  
print(age)
```

Printing with Text

```
print("My name is", name)
```

f-String Formatting

Used to insert variables directly inside a string.

Example:

```
name = "Om"  
age = 50  
mobile_num = 923184309218  
  
print(f"My name is {name} and my age is {age} and my mobile num is  
{mobile_num}")
```

5. List

What is a List?

A list is a collection data type used to store multiple values in a single variable.

Lists are:

- Ordered
- Mutable (changeable)
- Allow duplicate values
- Can store different data types

Example:

```
l = [1,2,3,4]
```

Why Use Lists?

Without lists:

```
a = 10  
b = 20  
c = 30
```

With lists:

```
numbers = [10,20,30]
```

Lists make data handling easier.

Properties of Lists

Property	Description
Ordered	Values keep their position
Mutable	Values can be changed
Indexed	Access using index numbers
Allows Duplicates	Same values can exist

(From: 4_list.ipynb)

What is List?

A list stores multiple values in one variable.

Example:

```
l = [1,2,3,4,5]
```

Indexing

Index	Value
0	1
1	2
2	3

Example:

```
l[0]
```

Slicing

Syntax:

```
list[start:end:jump]
```

Examples:

```
l[:]  
l[::1]  
l[::2]
```

Important Examples

Skip Values

```
l2 = [1,2,3,4,5,6,7,8,9,10]
```

```
l2[1::2]
```

Output:

```
[2,4,6,8,10]
```

Range Selection

```
l2[2:8]
```

Output:

```
[3,4,5,6,7,8]
```

6. Set

What is a Set?

A set is a collection data type used to store unique values.

Sets:

- Remove duplicate values
- Are unordered
- Do not support indexing
- Use curly brackets {}

Example:

```
s = {1,2,3}
```

Why Use Sets?

Sets are useful when duplicate values are not needed.

Example:

```
marks = {90,90,80,70}
```

Output:

```
{90,80,70}
```

Properties of Sets

Property	Description
Unique Values	Duplicates removed
Unordered	No fixed order
Mutable	Can add/remove items
No Indexing	Cannot access using index

(From: 5_data_structures_2.ipynb)

What is Set?

Set stores unique values only.

Example:

```
s = {1,1,1,2,2,3,4}
```

Output:

```
{1,2,3,4}
```

Duplicate values are automatically removed.

Set Example with Strings

```
s3 = {'a', 'a', 'a', 'b', 'b'}
```

Output:

```
{'a', 'b'}
```

7. Important Exam Concepts

Difference Between List and Tuple

List	Tuple
Mutable	Immutable
Uses []	Uses ()
Can change values	Cannot change values

Example:

```
l = [1,2,3]

# tuple
t = (1,2,3)
```

Difference Between List and Set

List	Set
Allows duplicates	Removes duplicates
Ordered	Unordered
Uses []	Uses {}

8. Common Errors

Division by Zero

```
5 / 0
```

This gives:

```
ZeroDivisionError
```

Type Error

```
"Hello" + 7
```

This gives:

```
TypeError
```

9. Most Important Practice Programs

Program 1: Check Datatype

```
x = 5  
print(type(x))
```

Program 2: Addition

```
a = 10  
b = 20  
  
print(a + b)
```

Program 3: List Slicing

```
l = [1,2,3,4,5,6]
print(l[:2])
```

Program 4: Set Duplicate Removal

```
s = {1,1,1,2,2,3}
print(s)
```

10. Teacher Pattern Analysis

Your teacher is mainly focusing on:

- Basic Python syntax
- Datatypes
- Operators
- Printing methods
- List slicing
- Sets and duplicates
- Beginner logic building
- Practical examples instead of theory only

Most important topics for exams/practice: 1. Data Types 2. Operators 3. List Slicing 4. String Concatenation 5. f-String 6. Set 7. Errors in Python

11. Revision Short Notes

Data Types

- int → whole number
- float → decimal number
- string → text
- bool → True/False

Operators

- ◦ add

- ◦ subtract
- ◦ multiply
- / division
- // floor division
- % remainder
- ** power

List

- Uses []
- Ordered
- Mutable

Set

- Uses {}
 - Removes duplicates
 - Unordered
-

12. Tips for Practical Exam

1. Always use proper indentation.
 2. Remember brackets:
 3. List → []
 4. Tuple → ()
 5. Set → {}
 6. Read errors carefully.
 7. Practice list slicing daily.
 8. Learn operator outputs.
 9. Practice print() formatting.
-

13. Suggested Next Topics to Practice

After these notebooks, you should practice:

- if-else
- for loop
- while loop
- functions
- dictionary

- pattern programs
 - mini projects
-

Final Conclusion

The repository is mainly designed for Python beginners and BCA 1st-year students. The notebooks focus more on understanding concepts through small examples and practical execution in Jupyter Notebook.